CS 5112 Fall 2025
Name: Anirudha Hadagali
NetID: ah2495
Collaborators: Fill Here

HW0: *Getting Comfortable With Python*
**Due: September 2, 11:59 PM ET**

**Problem 1.**

(a) (*1 point*) ()

Answer: *None*

(b) (*2 points*) ()

Answer: *None*

(c) (*2 points*) ()

Answer: *None*

(d) (*1 point*) ()

Answer: *None*

(e) (*4 points*) ()

Answer: *None*

**Problem 2.**

(a) (*1 point*) ()

Answer: *None*

(b) (*1 point*) ()

Answer: *None*

(c) (*1 point*) ()

Answer: *None*

(d) (*1 point*) ()

Answer: *None*

**Problem 3.**

(a) (*4 points*) ()

Answer: *None*

**Problem 4.**

(a) (*2 points*) ()

Answer: *None*

(b) (*1 point*) ()

Answer: *None*

**Problem 5.**

(a) (*2 points*) ()

Answer: This question tasked us to compute the factorial of 'n' using a python function 'reduce'. This was my first time using reduce, so it took some time for me to understand the use cases and the operations available.

From what I can understand about the 'reduce' function, it solves a smaller problem, and 'adds' up all the results together, iterating over some kind of list. This process is repeated multiple times until the end of the list is reached. At the end, all smaller results are squashed into a single answer. So it passes the result of the previous execution as the input of the next one.

So I think this method basically runs a function inside a for loop, with the bounds being the elements in the list passed as input. Since it is a for loop, this method runs in O(n) time. But this is only true if the operation you intend to do on the numbers is constant (multiplication, addition, subtraction, division, etc). But if the method was to sort, or filter, or any more complex function, then the time complexity would dramatically increase.

(b) (*2 points*) ()

Answer: This is a classic recursion problem to solve factorial. Factorial means multiply the current number and the number less than multiple times, until you reach one. You combine all the results to get the answer.

So for the base case, you have to check if the current number is less than or equal to 1. If it is, then you return 1. So the function accepts a number as a parameter, which means that after the base case is checked, you can multiply you number times the recursive function, but you pass in the number minus one. This means that the stack trace goes all the way down to 1, then multiplies its way up the stack, back to the original number.

The time complexity is O(n), because each multiplication step is constant O(1) time, and you exactly call that step n times. So it results in O(n) time overall for the recursive solution.